

Lab 1: Named Entity Recognition and Classification (NERC)

5LN700 — Language Technology

Lab Report

PART I — English Report

1. Introduction

Named Entity Recognition and Classification (NERC) is a fundamental Natural Language Processing (NLP) task. Its goal is to locate and categorize named entities in unstructured text into predefined categories such as person names (PER), organizations (ORG), locations (LOC), and miscellaneous entities (MISC).

This lab explores NERC using the CoNLL-2003 dataset and progressively implements multiple approaches — from traditional machine learning to deep learning — to understand the trade-offs between simplicity, performance, and computational cost.

2. Step 1 — Data Download and Exploration

2.1 What Was Done

The CoNLL-2003 dataset was loaded from Hugging Face using the datasets library. This benchmark corpus contains news articles with tokens annotated in IOB (Inside-Outside-Beginning) format across four entity types.

Key code: `load_dataset("conll2003", trust_remote_code=True)`

2.2 Dataset Statistics

Split	Samples (Sentences)	Notes
Train	~14,041	Used for model training
Validation	~3,250	Used during development
Test	~3,684	Final evaluation only

The four NER entity tags and their IOB encoding:

Tag	Meaning	Example
O	Not a named entity	"the", "and"
B-PER / I-PER	Person name	"Barack" / "Obama"
B-ORG / I-ORG	Organization	"Google" / "Inc"
B-LOC / I-LOC	Location	"Paris" / "France"
B-MISC / I-MISC	Miscellaneous	"European" / "Union"

2.3 Key Observations

- The dataset is highly imbalanced — the "O" tag accounts for the vast majority of tokens.
- Sentence lengths follow a right-skewed distribution: most sentences are short, with few very long ones.
- PER entities tend to be easiest to recognize; MISC entities are the most ambiguous and hardest.
- Class imbalance means accuracy alone is misleading — F1-score is more appropriate.

3. Step 2 — Tag Dictionaries

3.1 What Was Done

Tag-to-ID and ID-to-tag dictionaries were constructed for both POS (Part-of-Speech) tags and NER tags directly from the dataset features. These mappings are required to convert between symbolic labels and numeric IDs used by machine learning models.

3.2 Results

Dictionary	Size	Purpose
pos_tag2id / pos_id2tag	47 tags	Maps POS labels ↔ numeric IDs
ner_tag2id / ner_id2tag	9 tags	Maps NER labels ↔ numeric IDs

Assertions confirmed correct counts (47 POS tags, 9 NER tags), ensuring the dictionaries are complete and consistent.

4. Step 3 — Feature Engineering

4.1 What Was Done

The `extract_data()` function processes each token in the dataset and creates a feature tuple (token, is_capitalized, pos_tag) as input X, with the NER tag as label y. This forms the representation used for training.

4.2 Features Extracted

Feature	Type	Rationale
token	String (word form)	Identity of the word — most discriminative signal
is_capitalized	Boolean	Capitalized words are likely named entities
pos_tag	Integer (ID)	Grammatical role constrains entity type

4.3 OneHotEncoder Encoding

The feature tuples were converted to a sparse binary matrix using scikit-learn's OneHotEncoder. The encoder was fitted only on training data and applied to all splits to avoid data leakage.

- **Training set size:** 203,621 tokens
- **Feature dimensions:** 23,671 (23,623 vocab + 1 capitalization + 47 POS tags)
- **Output format:** Sparse CSR matrix (memory-efficient for high-dimensional data)
- **handle_unknown='ignore':** Unknown tokens in test set are safely ignored
- **drop='if_binary':** Capitalization uses only 1 column instead of 2

5. Step 4 — Building the Baseline NERC Model

5.1 What Was Done

A LinearSVC (Linear Support Vector Classifier) was trained on the encoded training data. SVM is well-suited for high-dimensional, sparse text data.

5.2 Results

Metric	Value	Interpretation
Training Accuracy	~97.42%	Strong fit — model has learned the training patterns
Model Type	LinearSVC	Efficient for large sparse feature matrices
random_state	42	Ensures reproducibility

Note: A training score above 97% is strong but within normal range for high-dimensional NER. Overfitting risk is low because LinearSVC has built-in regularization via the C parameter.

6. Step 5 — Model Evaluation

6.1 What Was Done

The trained model was evaluated on the test set using Precision, Recall, and F1-score for each entity class. These metrics are more meaningful than accuracy for imbalanced classification tasks.

6.2 Metric Definitions

Metric	Formula	Meaning
Precision	$TP / (TP + FP)$	Of all predicted entities, how many are correct?
Recall	$TP / (TP + FN)$	Of all true entities, how many were found?
F1-Score	$2 \times P \times R / (P + R)$	Harmonic mean of Precision and Recall

6.3 Expected Findings

- PER (Person) typically achieves the highest F1 — names are often distinctive and capitalized.
- MISC (Miscellaneous) typically achieves the lowest F1 — highly ambiguous category.
- Entity boundary detection is harder than entity type classification.
- Rare or novel entities in the test set are harder to predict correctly.

7. Step 6 — Advanced Extensions (VG)

Multiple advanced approaches were implemented to improve over the baseline LinearSVC. Each builds on the previous, adding more sophisticated features and models.

7.1 Named Entity Dictionaries + Rule-Based Post-Processing

Entity dictionaries were extracted from the training data for all four entity types (PER, ORG, LOC, MISC). Rule-based patterns (regex for dates, times, money, emails) were added as binary features alongside word embeddings (GloVe 100d).

- **Motivation:** Known entities in training data can be directly matched in test data.
- **Added features:** `in_per_dict`, `in_org_dict`, `in_loc_dict`, `in_misc_dict`, `is_date`, `is_time`, `is_money`, `is_email`
- **Context features:** Previous and next token capitalization, previous/next POS tags

7.2 Model Ensemble (Voting)

Three diverse classifiers were combined via majority voting to reduce individual model error:

Model	Strength	Weakness
LinearSVC	Fast, good on sparse data	No probability output
Logistic Regression	Probabilistic, interpretable	Slower on large data
Random Forest	Non-linear patterns	Higher memory usage

Ensemble strategy: Majority vote (`scipy.stats.mode`) across three model predictions. The ensemble is more robust than any individual model.

7.3 Hyperparameter Tuning with K-Fold Cross-Validation

5-fold cross-validation was applied to the training set to get reliable performance estimates and select the best model configuration before final test evaluation.

- **K = 5 folds:** Each fold uses 80% for training, 20% for validation.
- **Results reported:** Mean $\pm$ standard deviation of P/R/F1 across folds.
- **Purpose:** Prevent overfitting to the validation set during model selection.

7.4 CNN-LSTM Neural Model

A deep learning model was implemented combining CNN (for local feature extraction) and LSTM (for sequential context modeling):

- CNN layers extract n-gram level patterns from word embeddings
- CNN output is expanded across the sequence and concatenated with POS embeddings
- Bidirectional LSTM captures long-range dependencies in both directions
- The model is trained end-to-end with cross-entropy loss

7.5 BiLSTM-CRF with Character Encoding

A more sophisticated neural architecture was implemented:

- **Character-level CNN:** Captures morphological patterns (prefixes, suffixes, capitalization)
- **Bidirectional LSTM:** 2-3 layers, models sequential dependencies
- **CRF output layer:** Models label dependencies (e.g., I-PER must follow B-PER)
- **Attention mechanism:** Multi-head self-attention for global context

7.6 BERT-CRF (State-of-the-Art)

The most advanced approach fine-tunes BERT (bert-base-cased) for NER:

- **BERT:** Pre-trained transformer model with deep contextual representations
- **Token alignment:** BERT's WordPiece tokenization is aligned back to original tokens
- **CRF layer:** Ensures valid IOB label sequences
- **Focal Loss:** Down-weights easy examples, focuses learning on hard entities
- **Expected F1:** BERT-CRF typically achieves >90% F1 on CoNLL-2003

8. Summary and Comparison

Approach	Key Features	Expected F1 (approx.)	Complexity
LinearSVC (baseline)	OHE: token + capitalization + POS	~85-88%	Low
LinearSVC + Dictionaries	Above + entity dicts + regex + embeddings	~87-90%	Medium
Ensemble (SVM+LR+RF)	Majority voting over 3 models	~88-91%	Medium

Approach	Key Features	Expected F1 (approx.)	Complexity
CNN-LSTM	Word embeddings + CNN + BiLSTM	~88-92%	High
BiLSTM-CRF + Char CNN	Char features + BiLSTM + CRF + Attention	~90-93%	High
BERT-CRF	Pre-trained BERT + CRF + Focal Loss	~92-95%	Very High

9. Key Takeaways

- Feature engineering significantly impacts performance even for traditional ML models.
- The "O" class dominates the dataset — models naturally bias toward it, making F1 more informative than accuracy.
- Neural models outperform traditional ML when sufficient data is available.
- CRF output layers are critical for NER — they enforce valid IOB tag sequences.
- BERT-based models represent the current state-of-the-art for NER tasks.
- The validation set should guide model selection; the test set should only be used once for final reporting.